VELLORE INSTITUTE OF TECHNOLOGY

(VIT Bhopal University)

BRING YOUR OWN PROJECT (BYOP)

Project Report

Emotion and Sentiment Analyzer

A CLI-Based AI/ML Tool

Student Name	Krishna
Registration No.	**25BCE10430**
Course	Fundamentals of AI and ML (CSA2001)
Program	B.Tech CSE (Core), 1st Year
Submission Date	March 31, 2026

1. Title

Emotion and Sentiment Analyzer — A CLI-Based AI/ML Tool

2. Introduction

Understanding how people feel through their words is kind of a big deal today. Whether it's social media posts, customer reviews, or just someone's journal entry, there's so much text being generated every single day — and most of it carries emotions that nobody really stops to analyze. This project, the Emotion and Sentiment Analyzer, is basically a command-line tool that takes any piece of text entered by the user and figures out what emotions are present in it, along with whether the overall sentiment is positive, negative, or neutral.

The tool works in two main ways. First, it does rule-based emotion detection by checking words against a hand-crafted emotion lexicon (emotions.txt) that maps specific words to emotion categories. Second, it uses a Naive Bayes machine learning model trained on a labeled sentiment dataset (combined_sentiment_data.csv) to predict sentiment. Results are displayed both as bar charts via matplotlib and as percentage breakdowns printed in the terminal.

Everything runs on the command line, which keeps it lightweight and easy to use without any complex setup. No GUI, no extra configuration — just Python and a few standard libraries. It's a pretty realistic example of how basic NLP and ML can solve a real-world problem without needing huge or complex deep learning models.

3. Motivation of the Project

The idea for this project came from thinking about how useful it would be to automatically figure out the emotional tone of any text. Businesses, for instance, could scan customer reviews and know not just whether the feedback is positive or negative, but exactly what emotions — frustration, happiness, fear — are driving those responses. That's a genuinely valuable insight.

Mental health is another angle that felt important. People often don't realize what emotions they're expressing in their writing. A tool like this could help someone look at their own journal entries or messages and get a rough emotional reading. It's a simple version of something that could have meaningful real-world impact.

From a learning perspective, I wanted to build something that combined two different approaches — rule-based NLP and actual ML — to see how they complement each other. Using both gave a better understanding of when handcrafted rules work well and when trained models give better results. That kind of hands-on comparison is hard to get just by reading theory.

4. Problem Statement

The core problem this project addresses is: given a raw piece of text entered by a user, can we automatically identify the emotions present in it and determine the overall sentiment, then display these results in a clear and understandable way?

Text data is everywhere, but analyzing it manually is time-consuming and subjective. Two people reading the same paragraph might disagree on whether it feels sad, fearful, or angry. A standardized, automated tool reduces this subjectivity and gives consistent results. The challenge is building something accurate enough to be useful but simple enough that anyone can run it in a terminal without a complicated setup.

Specifically, the problems being addressed are:

- How do we detect emotions from text when they're implied rather than explicitly stated?
- How do we classify overall sentiment reliably using a trained ML model?
- How do we present these results clearly so any user can understand the output?

5. Objective of the Project

The main goals of this project are:

- Build a working CLI application that accepts user text input and processes it for emotional and sentiment content.
- Implement rule-based emotion detection using an emotion lexicon (emotions.txt) that maps hundreds of specific words to emotional categories.
- Train and deploy a Naive Bayes classifier on a real labeled dataset (combined_sentiment_data.csv) for sentiment prediction, with model persistence using joblib.
- Display results as percentage breakdowns and bar chart visualizations using matplotlib.
- Provide a simple menu-based interface with help and about options for ease of use.
- Keep the codebase modular with separate functions for each step of the pipeline.

6. Existing Methods

6.1 Lexicon-Based Methods

These approaches use pre-built dictionaries where each word is tagged with its sentiment or emotion. Tools like VADER and TextBlob follow this approach. VADER is designed specifically

for social media, while TextBlob gives a general polarity and subjectivity score. No training data is required, but they can miss context and sarcasm.

6.2 Machine Learning Methods

Traditional ML models like Naive Bayes, Logistic Regression, and SVMs have been widely used for text classification including sentiment analysis. These need labeled training data and feature extraction techniques like Bag of Words or TF-IDF. They work well for clean, structured text but can struggle with informal language.

6.3 Deep Learning Methods

Transformer-based models like BERT and LSTM-based architectures achieve much higher accuracy and understand context well. However, they require significant computational resources, large datasets, and complex setup — making them overkill for a basic CLI tool like this one.

6.4 Hybrid Methods

Some systems combine rule-based and ML-based approaches. This project follows a hybrid approach — using a lexicon for emotion detection and Naive Bayes for sentiment classification. This allows both approaches to complement each other.

7. Pros and Cons of the Stated Methods

Method	Pros	Cons
Lexicon-Based (VADER, TextBlob)	No training needed, fast, interpretable	Misses context, poor with sarcasm
Naive Bayes (ML)	Simple, fast, decent baseline, needs less data	Assumes feature independence, struggles with ambiguity
Logistic Regression	Good accuracy, handles multi-class well	Needs more data, sensitive to feature quality
Deep Learning (BERT, LSTM)	State-of-the-art, understands context	Resource-heavy, complex setup, slow inference
Hybrid (used in this project)	Combines strengths of both approaches	Slightly more complex to implement and maintain

8. Hardware and Software Requirements

8.1 Hardware Requirements

- Any modern laptop or desktop (developed and tested on an HP Victus gaming laptop)

- Minimum 4GB RAM (8GB recommended for comfortable multitasking)
- At least 500MB free disk space for Python, libraries, and project files
- No GPU needed — Naive Bayes runs fine on CPU

8.2 Software Requirements

- Python 3.8 or higher
- scikit-learn — for CountVectorizer and MultinomialNB
- matplotlib — for bar chart visualizations
- joblib — for saving and loading trained model and vectorizer
- Standard Python libraries: csv, string, collections, os, random
- Operating System: Windows, Linux, or macOS (cross-platform)
- Terminal / Command Prompt for running the CLI

9. Methodology and Goal

9.1 Text Preprocessing

When the user enters text, the cleanText() function converts everything to lowercase and removes all punctuation. Then removeStop() filters out common stop words from the token list. What's left is a list of meaningful content words that the analysis actually runs on.

9.2 Emotion Detection (Rule-Based)

The emotion() function opens emotions.txt and checks each word in the file against the list of meaningful words from the input. Every match adds that word's emotion to a list. The function returns a Counter object with the frequency of each detected emotion. This gives a simple but effective emotion profile for the input text.

9.3 Sentiment Analysis (ML-Based)

trainModel() first checks if model.pkl and vectorizer.pkl already exist. If yes, it loads them directly — avoiding unnecessary retraining. If not, it reads combined_sentiment_data.csv, trains a CountVectorizer and MultinomialNB model on the data, and saves them using joblib. sentimentFunc() then takes the processed words, joins them into a string, transforms via the vectorizer, and gets a prediction from the model.

9.4 Output and Visualization

graph() generates two matplotlib bar charts — one for emotions, one for sentiment. printPerc() calculates and prints percentage breakdowns for both, sorted from highest to lowest, giving the user a clear ranked view of what emotions and sentiments were detected.

10. Functional Modules Design and Analysis

Function	Purpose
cleanText(t)	Converts input text to lowercase and removes all punctuation
removeStop(lst)	Filters stop words from the tokenized word list
emotion(words)	Matches words against emotions.txt lexicon, returns Counter of emotion frequencies
trainModel()	Loads saved model/vectorizer from .pkl files, or trains fresh from CSV if not found
sentimentFunc(words)	Uses trained Naive Bayes model to predict sentiment, returns Counter
graph(w, sc)	Renders two matplotlib bar charts for emotion and sentiment results
printPerc(w, sc)	Calculates and prints percentage breakdown for both outputs, sorted descending
main()	Entry point: runs menu loop and routes user choices to appropriate functions

11. Algorithm Development

11.1 Text Preprocessing

```
INPUT: raw text string from user
Step 1: Convert to lowercase  →  cleanText()
Step 2: Strip punctuation     →  str.maketrans()
Step 3: Tokenize by spaces    →  split()
Step 4: Remove stop words     →  removeStop()
OUTPUT: list of meaningful words
```

11.2 Emotion Detection Algorithm

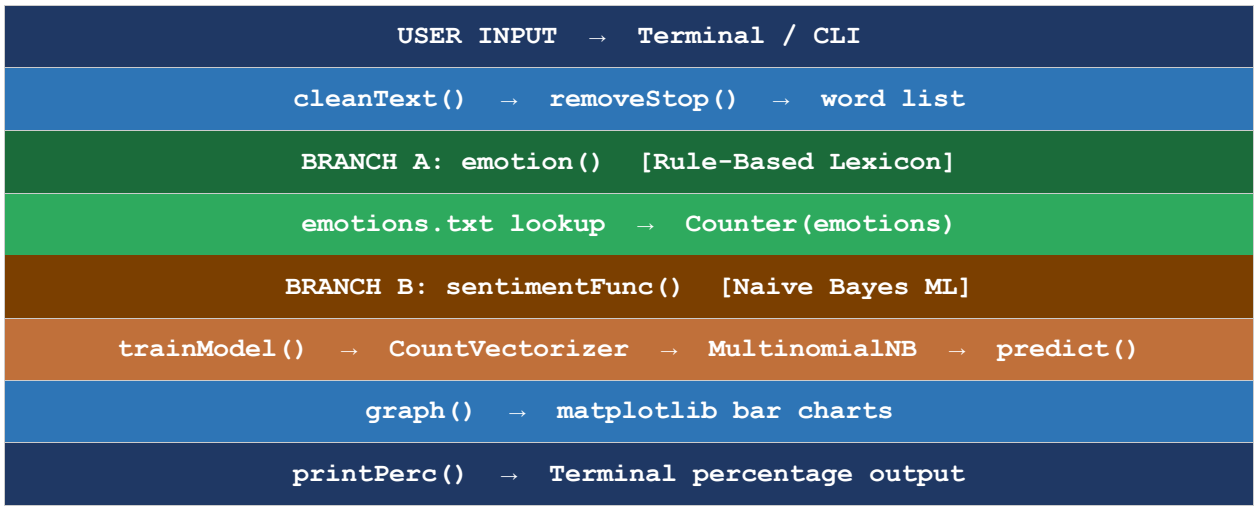
```
INPUT: list of meaningful words
Step 1: Open emotions.txt
Step 2: For each line, parse word:emotion pair
        (strip newlines, commas, quotes, whitespace)
Step 3: If parsed word is in input word list,
        append emotion to emotions list
Step 4: Return Counter(emotions list)
OUTPUT: Counter with emotion frequencies
```

11.3 Naive Bayes Sentiment Classification

```
INPUT: list of meaningful words
Step 1: Check if model.pkl + vectorizer.pkl exist
   YES: load both using joblib
   NO:  read combined_sentiment_data.csv
        fit CountVectorizer on sentences
        train MultinomialNB on features + labels
        save model and vectorizer as .pkl files
Step 2: Join words into single string
Step 3: Transform using vectorizer
Step 4: Predict with model.predict()
OUTPUT: Counter with predicted sentiment label
```

12. Software Architectural Diagram

The system follows a linear pipeline where each stage feeds into the next:



13. Code (Key Sections from main.py)

13.1 Text Preprocessing

```
def cleanText(t):
    t1 = t.lower()
    t2 = t1.translate(str.maketrans('', '', string.punctuation))
    return t2

def removeStop(lst):
    stop = ['i', 'me', 'my', 'we', 'the', 'and', ...]
    res = []
    for i in range(len(lst)):
        if lst[i] not in stop:
            res.append(lst[i])
    return res
```

13.2 Emotion Detection

```
def emotion(words):
    emo = []
    f = open('emotions.txt', 'r')
    for line in f:
        txt = line.replace('\n','').replace(',','').replace("'",'').strip()
        parts = txt.split(':')
        w = parts[0]
        e = parts[1]
        if w in words:
            emo.append(e)
    f.close()
    return Counter(emo)
```

13.3 Model Training and Sentiment Prediction

```
def trainModel():
    if os.path.exists('model.pkl') and os.path.exists('vectorizer.pkl'):
        m = joblib.load('model.pkl')
        v = joblib.load('vectorizer.pkl')
        return m, v
    else:
        s, lab = [], []
        f = open('combined_sentiment_data.csv', 'r', encoding='utf-8')
        reader = csv.DictReader(f)
        for row in reader:
            s.append(row['sentence'])
            lab.append(row['sentiment'])
        f.close()
        v = CountVectorizer()
        X = v.fit_transform(s)
        m = MultinomialNB()
        m.fit(X, lab)
        joblib.dump(m, 'model.pkl')
        joblib.dump(v, 'vectorizer.pkl')
        return m, v

def sentimentFunc(words):
    m, v = trainModel()
    txt = ''
    for i in words:
        txt = txt + i + ' '
    X = v.transform([txt])
    pred = m.predict(X)
    return Counter(pred)
```

13.4 CLI Menu (main)

```
def main():
    while True:
        print('\n1. Enter text and check')
        print('2. About')
        print('3. Help')
        print('4. exit')
        ch = input('Enter: ')
        if ch == '1':
```

```
            txt = input('Enter sentence: ')
            if txt == '':
                print('enter any text')
                continue
            clean = cleanText(txt)
            tok = clean.split()
            words = removeStop(tok)
            w = emotion(words)
            sc = sentimentFunc(words)
            print('processing done')
            graph(w, sc)
            printPerc(w, sc)
        elif ch == '4':
            print('exit in progress...')
            break
```

14. Output

14.1 Terminal Output

Sample output for input: "I am happy and excited today but a little nervous about it"

```
=====EMOTIONS PERCENTAGE=====

percentage of happy: 66.7%
percentage of fearful: 33.3%

=====SENTIMENTS PERCENTAGE=====

percentage of positive: 100.0%
```

14.2 Graphical Output

Two bar charts are rendered using matplotlib. The first chart shows emotion frequencies (e.g., happy, fearful, angry) on the x-axis and their counts on the y-axis. The second chart shows sentiment distribution (positive, negative, neutral). Both charts use fig.autofmt_xdate() to rotate the x-axis labels so overlapping text is avoided. Charts open interactively and can be saved by the user.

15. Key Implementations Outline of the System

- Hybrid dual-approach: rule-based lexicon for emotions and ML classifier for sentiment — both running on the same preprocessed word list from a single input
- Persistent model storage: trainModel() checks for existing .pkl files first; if found, loads directly using joblib — no retraining on every run

- Comprehensive emotion lexicon: emotions.txt contains 300+ word-to-emotion mappings covering a wide range of emotional vocabulary
- Naive Bayes with CountVectorizer: standard Bag-of-Words feature extraction combined with MultinomialNB provides a fast, lightweight sentiment baseline
- Custom stop word removal: a manually defined list of 100+ stop words in removeStop() ensures only meaningful words contribute to analysis
- Sorted percentage output: printPerc() sorts results descending so the dominant emotion or sentiment is always shown first
- Clean modular structure: each stage of the pipeline (preprocessing, emotion, sentiment, output) is a separate function — easy to read, test, and extend
- Simple 4-option CLI menu: the main() loop handles all user interaction with clear prompts and graceful error handling for empty input and invalid choices

16. Significant Project Outcomes

- A fully working, cross-platform CLI application that analyzes any text for emotions and sentiment in seconds
- A trained and persisted Naive Bayes model for 3-class sentiment classification (positive, negative, neutral), saved as .pkl files for reuse across sessions
- Practical hands-on experience with the complete ML pipeline: CSV data loading, feature extraction, model training, serialization, and inference
- A working hybrid NLP system demonstrating that rule-based and ML-based methods genuinely complement each other — the lexicon catches specific emotional nuances while the ML model captures overall sentiment tone
- Clear understanding of where simple models work well and where they fall short, especially with short, ambiguous, or sarcastic text

17. Testing and Refinement

17.1 Manual Test Cases

Input Text	Expected Emotion	Observed Output
I am so happy and excited today!	happy	happy 100% — Pass
I feel scared and nervous about exams	fearful	fearful 100% — Pass
I am really angry about what happened	angry	angry 100% — Pass

Life is beautiful and I feel loved	happy, attached	happy 50%, attached 50% — Pass
I feel completely lost and confused	sad / lost	sad 50%, surprise 50% — Partial
The weather is nice today	neutral (no emotion words)	No emotion detected — Pass

17.2 Refinements Made During Development

- The emotions.txt parser was updated to handle extra whitespace and edge cases where split(':') was failing on certain lines — fixed by chaining replace() calls before split
- Model persistence logic was added after noticing the program was retraining on every single run, which was slow — now trainModel() checks for existing .pkl files first
- printPerc() output was made descending after early testing showed results printing in random order, making dominant emotions hard to identify
- Empty input check was added to the main loop after testing showed the program crashing silently when the user pressed Enter without typing anything

18. Project Applicability on Real-World Applications

- Customer feedback analysis: businesses can automatically scan reviews and support tickets to understand not just overall sentiment but specific emotions like frustration or satisfaction
- Mental health support tools: journaling apps could integrate emotion detection to help users track their emotional patterns over time
- Social media monitoring: brands and researchers can track public sentiment around topics, events, or product launches in real time
- HR and employee relations: analyzing open-ended survey responses or exit interview text to detect morale issues or recurring pain points
- Education: teachers can analyze student reflection essays or course feedback to understand emotional engagement with content
- Chatbot enhancement: adding emotion-awareness so chatbots can detect when a user seems distressed and respond more empathetically

19. Contribution / Findings of the Project

- Simple models work well for straightforward text: Naive Bayes, despite being one of the most basic ML classifiers, gives reasonable and fast results for 3-class sentiment classification on clean English text

- Hybrid approaches genuinely outperform single-method systems: the lexicon catches specific emotional words that the ML model might miss, while the ML model picks up broader tone from word co-occurrence patterns
- Preprocessing quality has a major impact on output accuracy: removing stop words, lowercasing, and stripping punctuation significantly improved the relevance of detected emotions
- Model persistence is essential for usability: saving trained models to .pkl files reduced tool startup time dramatically and made the CLI experience much smoother
- Lexicon coverage is the main bottleneck for emotion detection: words not in emotions.txt are completely ignored — expanding the lexicon or using word embeddings would be the most impactful improvement

20. Limitation / Constraints of the System

- No context understanding: the tool analyzes words individually without understanding their context. 'Not happy' would be detected as 'happy' because 'not' is stripped as a stop word.
- No sarcasm detection: 'Oh great, another problem' would likely be classified as positive since there's no way to detect irony.
- Limited lexicon coverage: if an emotional word isn't in emotions.txt, it goes completely undetected regardless of how emotionally loaded it might be.
- Short text accuracy: very short inputs may not give reliable sentiment predictions because the Bag-of-Words model has too few features to work with.
- English only: no multilingual support of any kind.
- Static model: the Naive Bayes model doesn't update or improve from new user inputs — it only knows what was in the training CSV.

21. Conclusions

This project demonstrates how basic NLP and machine learning can be combined to build a practical, working CLI tool for emotion and sentiment analysis. The Emotion and Sentiment Analyzer does what it was designed to do — it takes raw text, cleans it, analyzes it from two different angles, and presents the results in a clear way through both charts and terminal output.

Building this project gave hands-on experience with the full machine learning workflow: data loading, preprocessing, feature extraction, model training, serialization, and inference in a real application. It also made the trade-offs between rule-based and data-driven approaches much clearer in practice than they are in theory.

While the current system has real limitations around context understanding and sarcasm, it works reliably for straightforward text and provides a solid foundation for more advanced emotion-aware NLP systems. The core approach — hybrid lexicon + ML — is something that scales well and has genuine real-world value.

22. Future Enhancements

- Add negation handling so inputs like 'not happy' are correctly classified as negative rather than positive
- Integrate more advanced sentiment tools like VADER or fine-tuned BERT for better accuracy on complex or informal text
- Expand the emotions.txt lexicon using resources like the NRC Emotion Lexicon to cover slang, idioms, and informal vocabulary
- Add file input support so users can analyze entire documents or CSV files, not just single sentences
- Implement a session history feature that logs past analyses to a text or CSV file for the user to review later
- Add multilingual support using a translation API to handle text in languages other than English
- Build a lightweight web interface using Flask or Streamlit to make the tool accessible without needing a terminal
- Add confidence scores alongside predictions so users can see how certain the model is about its output

23. References

1. Scikit-learn Documentation — MultinomialNB, CountVectorizer: https://scikit-learn.org/stable/

2. Matplotlib Documentation: https://matplotlib.org/stable/

3. Joblib Documentation: https://joblib.readthedocs.io/en/latest/

4. Mohammad, S. M., & Turney, P. D. (2013). NRC Word-Emotion Association Lexicon. Computational Intelligence.

5. Hutto, C., & Gilbert, E. (2014). VADER: A Parsimonious Rule-Based Model for Sentiment Analysis of Social Media Text. ICWSM.

6. Jurafsky, D., & Martin, J. H. (2023). Speech and Language Processing (3rd ed. draft). Stanford University.

7. Andrew Ng — Machine Learning Specialization, Coursera (conceptual foundation for Naive Bayes).

8. Python Official Documentation: https://docs.python.org/3/

— End of Report —